

Министерство науки и высшего образования РФ
ФГАОУ ВО «Омский государственный технический университет»
Кафедра «Автоматизированные системы обработки информации и управления»

ОТЧЁТ
по лабораторным работам
студента Нурсалимова Данияра Ерлановича ПИ-221

Ассистент

Л.В. Шапова

Студент

Д. Е. Нурсалимов

Омск 2024

Лабораторная работа 1

Задание: Реализовать паттерн проектирования «Prototype» («Прототип») на примере класса «Файл».

Код выполнения:

```
public class Main {
    public static void main(String[] args)
    {
        Prototype Domen1 = new Domens(".ru");
        Prototype Domen2 = Domen1.clone();

        System.out.println(Domen1.getAdress());
        System.out.println(Domen2.getAdress());

        Prototype miniDomen1 = new miniDomens(".com");
        Prototype miniDomen2 = miniDomen1.clone();

        System.out.println(miniDomen1.getAdress());
        System.out.println(miniDomen2.getAdress());
    }
}
```

```
public class Domens implements Prototype {
    String adress;
    public Domens(String address) {this.adress = address;}

    public Prototype clone() {return new Domens(this.adress);}
    @Override
    public String getAdress() {
        return adress;
    }
}
```

```
public class miniDomens implements Prototype {
    String adress;
    public miniDomens(String address) {this.adress = address;}

    @Override
    public Prototype clone() {return new miniDomens(this.adress);}
    public String getAdress() {return adress;}
}
```

```
public interface Prototype {
    Prototype clone();
    String getAdress();
}
```

Лабораторная работа 2

Задание: Реализовать паттерн «Строитель» на примере поэтапного создания объекта «Робот».

Код выполнения:

```
public class Main {
    public static void main(String[] args) {
        RobotBuilder robotBuilder = new ConcreteRobotBuilder(); //Создаём
строителя
        Director director = new Director(robotBuilder); //Создаём директора

        Robot robot = director.constructRobot(); //Директор вызывает метод
constructRobot(), который пошагово строит объект "Робот" с помощью строителя
        System.out.println(robot); //Результат выводится на экран
    }
}
```

```
// Директор, который управляет процессом построения робота
class Director {
    //принимает на вход объект типа RobotBuilder
    private RobotBuilder robotBuilder;

    public Director(RobotBuilder robotBuilder) {
        this.robotBuilder = robotBuilder;
    }
    //вызывает методы для построения различных частей робота
    public Robot constructRobot() {
        robotBuilder.buildHead();
        robotBuilder.buildBody();
        robotBuilder.buildLegs();
        return robotBuilder.getRobot(); //возвращает готовый объект робота
    }
}
```

```
// Конкретный строитель роботов
//Создает объект Robot и реализует методы для построения головы, тела и ног
робота,
//а также метод для получения готового объекта робота
class ConcreteRobotBuilder implements RobotBuilder {
    private Robot robot;

    public ConcreteRobotBuilder() {
        this.robot = new Robot();
    }

    @Override
    public void buildHead() {
        robot.setHead("Металлическая голова");
    }

    @Override
    public void buildBody() {
        robot.setBody("Металлическое тело");
    }

    @Override
    public void buildLegs() {
        robot.setLegs("Металлические ноги");
    }
}
```

```
    @Override
    public Robot getRobot() {
        return this.robot;
    }
}
```

```
// Класс, представляющий объект "Робот"
class Robot {
    private String head;
    private String body;
    private String legs;
    //методы setHead, setBody, setLegs для установки значений частных полей
    public void setHead(String head) {
        this.head = head;
    }

    public void setBody(String body) {
        this.body = body;
    }

    public void setLegs(String legs) {
        this.legs = legs;
    }
    //возвращает строковое представление объекта
    @Override
    public String toString() {
        return "Robot{" +
            "head='" + head + '\'' +
            ", body='" + body + '\'' +
            ", legs='" + legs + '\'' +
            '}';
    }
}
```

```
// Интерфейс строителя роботов
interface RobotBuilder {
    //методы для построения различных частей робота
    void buildHead();
    void buildBody();
    void buildLegs();
    //метод getRobot для получения готового объекта робота
    Robot getRobot();
}
```

Лабораторная работа 3

Задание: Реализовать схему MVC на примере валютного калькулятора. Результаты расчета вывести в файл .txt.

Код выполнения:

```
public class Main {
    public static void main(String[] args) {
        CurrencyConverter model = new CurrencyConverter();// Создание объекта
        модели
        CurrencyCalculatorView view = new CurrencyCalculatorView();// Создание
        объекта вида
        CurrencyCalculatorController controller = new
        CurrencyCalculatorController(model, view);// Создание объекта контроллера с
        передачей модели и вида.

        double amount = 100;
        double exchangeRate = 1.1;
        // Установка суммы и обменного курса для конвертации

        controller.calculateCurrency(amount, exchangeRate);
        // Вызов метода контроллера для выполнения конвертации и отображения
        результата.
    }
}
```

```
// Класс, отвечающий за конвертацию валюты. Модель в MVC.
public class CurrencyConverter {
    public double convertCurrency(double amount, double exchangeRate) {
        // Метод для конвертации суммы по заданному обменному курсу
        return amount * exchangeRate;
    }
}

// Класс, отвечающий за управление логикой приложения. Контроллер в MVC.
public class CurrencyCalculatorController {
    private CurrencyConverter model;
    private CurrencyCalculatorView view;

    // Конструктор, принимающий модель и вид.
    public CurrencyCalculatorController(CurrencyConverter model,
    CurrencyCalculatorView view) {
        this.model = model;
        this.view = view;
        // Инициализация модели и вида.
    }

    // Метод для выполнения конвертации и отображения результата.
    public void calculateCurrency(double amount, double exchangeRate) {
        double result = model.convertCurrency(amount, exchangeRate);// Вызов
        метода модели для конвертации валюты
        view.displayResult(result);// Вызов метода вида для отображения
        результата.
    }
}
```

```
import java.io.FileWriter;
import java.io.IOException;
```

```
// Класс, отвечающий за отображение результата. Вид в MVC.
public class CurrencyCalculatorView {
    // Метод для отображения результата конвертации.
    public void displayResult(double result) {
        // Создание объекта для записи результата в файл txt.
        try (FileWriter writer = new FileWriter("result.txt")) {
            writer.write("Converted amount: " + result);
        } catch (IOException e) { // Запись результата в файл
            e.printStackTrace(); // Обработка исключения, если возникла ошибка при
// записи в файл.
        }
    }
}
```

Лабораторная работа 4

Задание к работе:

1. Разработать программу To Do (задачник в компьютере).
2. Разработать систему тестов для валидации разработанной программы.

ВВЕДЕНИЕ

В ходе работы была разработана программа To Do.

Для разработки программы были использованы среда IntelliJ IDEA, библиотеки `java.io.*`, `javax.swing.*`, `java.awt.*`, `java.util`, `org`, `jemmy`.

В качестве языка разработки был выбран высокоуровневый язык программирования Java.

НАЗНАЧЕНИЕ ПРОГРАММЫ

Разработанная программа предназначена для ведения списка задач, а также отметки выполнения их.

Для запуска необходимо запустить программу в Inteleji IDEA.

ОПИСАНИЕ ЯДРА ПРОГРАММЫ

Программа включает классы:

- Main
- Task
- TaskView
- TaskModelTest
- TaskController
- TaskControllerTest

Класс Main отвечает за запуск программы, он вызывает другие классы и связывает их с классом Controller.

Исходный код модуля приведён ниже.

```
public class Main {  
    public static void main(String[] args) {  
        // Создание модели  
        TaskModel model = new TaskModel();  
  
        // Создание контроллера и связывание его с моделью  
        TaskController controller = new TaskController(model);  
  
        // Запуск графического интерфейса пользователя в потоке обработки  
        // событий  
        javax.swing.SwingUtilities.invokeLater(new Runnable() {  
            public void run() {  
                // Создание представления и связывание его с контроллером  
                TaskView view = new TaskView(controller);  
  
                // Отображение пользовательского интерфейса  
                view.setVisible(true);  
            }  
        });  
    }  
}
```

Класс Task отвечает за данные программы.

Исходный код модуля приведён ниже.

```
import java.io.Serializable;  
import java.util.ArrayList;  
import java.util.List;  
  
// Класс задачи  
public class Task implements Serializable {  
    private String description;  
    private boolean isCompleted;  
  
    // Конструктор
```

```

    public Task(String description) {
        this.description = description;
        this.isCompleted = false;
    }

    // Геттер и сеттер для описания задачи
    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    // Геттер и сеттер для статуса задачи
    public boolean isCompleted() {
        return isCompleted;
    }

    public void setCompleted(boolean completed) {
        isCompleted = completed;
    }

    // Переопределение метода toString для удобного отображения задачи
    @Override
    public String toString() {
        return (isCompleted ? "[X] " : "[ ] ") + description;
    }
}

// Класс модели, содержащий список задач
class TaskModel implements Serializable {
    private List<Task> tasks;

    // Конструктор
    public TaskModel() {
        tasks = new ArrayList<>();
    }

    // Добавление новой задачи
    public void addTask(String description) {
        tasks.add(new Task(description));
    }

    // Удаление задачи
    public void removeTask(int index) {
        if(index >= 0 && index < tasks.size()) {
            tasks.remove(index);
        }
    }

    // Получение списка всех задач
    public List<Task> getTasks() {
        return tasks;
    }

    // Отметить задачу как выполненную
    public void completeTask(int index) {
        if(index >= 0 && index < tasks.size()) {
            tasks.get(index).setCompleted(true);
        }
    }

    // Получение количества задач

```

```

    public int getTaskCount() {
        return tasks.size();
    }

    // Получение количества невыполненных задач
    public int getRemainingTasks() {
        int count = 0;
        for(Task task : tasks) {
            if (!task.isCompleted()) {
                count++;
            }
        }
        return count;
    }
}

```

Класс TaskView отвечает за интерфейс приложения, содержит конструктор View. В конструкторе View описываются все элементы, которые будут показаны на экране. В методах описываются форматы фреймов и их расположение.

Исходный код модуля приведён ниже.

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.List;

public class TaskView extends JFrame {
    private TaskController controller;
    private DefaultListModel<String> taskListModel;
    private JList<String> taskList;
    private JTextField newTaskField;
    private JButton addButton;
    private JButton removeButton;
    private JButton completeButton;
    private JButton saveButton;
    private JButton loadButton;

    // Конструктор
    public TaskView(TaskController controller) {
        this.controller = controller;
        initializeComponents();
        layoutComponents();
        initializeListeners();
        updateTaskList();
    }

    // Инициализация компонентов
    private void initializeComponents() {
        taskListModel = new DefaultListModel<>();
        taskList = new JList<>(taskListModel);
        newTaskField = new JTextField(20);
        addButton = new JButton("Add");
        removeButton = new JButton("Remove");
        completeButton = new JButton("Complete");
        saveButton = new JButton("Save");
        loadButton = new JButton("Load");
    }

    // Расположение компонентов на фрейме

```

```

private void layoutComponents() {
    this.setLayout(new BorderLayout());
    this.add(new JScrollPane(taskList), BorderLayout.CENTER);

    JPanel inputPanel = new JPanel(new FlowLayout());
    inputPanel.add(newTaskField);
    inputPanel.add(addButton);
    inputPanel.add(removeButton);
    inputPanel.add(completeButton);
    inputPanel.add(saveButton);
    inputPanel.add(loadButton);
    this.add(inputPanel, BorderLayout.SOUTH);

    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.setTitle("Task Manager");
    this.pack();
    this.setVisible(true);
}

// Инициализация слушателей для кнопок и текстового поля
private void initializeListeners() {
    addButton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            controller.addTask(newTaskField.getText());
            newTaskField.setText("");
            updateTaskList();
        }
    });

    removeButton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            int selectedIndex = taskList.getSelectedIndex();
            if (selectedIndex != -1) {
                controller.removeTask(selectedIndex);
                updateTaskList();
            }
        }
    });

    completeButton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            int selectedIndex = taskList.getSelectedIndex();
            if (selectedIndex != -1) {
                controller.completeTask(selectedIndex);
                updateTaskList();
            }
        }
    });

    saveButton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            controller.saveTasksToFile("tasks.dat");
        }
    });

    loadButton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            controller.loadTasksFromFile("tasks.dat");
            updateTaskList();
        }
    });
}

// Обновление списка задач на экране

```

```

private void updateTaskList() {
    List<Task> tasks = controller.getTasks();
    taskListModel.clear();
    for (Task task : tasks) {
        taskListModel.addElement(task.toString());
    }
}
}

```

Класс TaskController отвечает за связь между TaskView и Task, содержит классы действий кнопок для всех TaskView. В классах вызываются необходимые методы, которые должны срабатывать при нажатии кнопки.

Исходный код модуля приведён ниже.

```

import java.io.*;
import java.util.List;

public class TaskController {
    private TaskModel model;

    // Конструктор
    public TaskController(TaskModel model) {
        this.model = model;
    }

    // Добавление новой задачи
    public void addTask(String description) {
        if (description != null && !description.trim().isEmpty()) {
            model.addTask(description);
        }
    }

    // Удаление задачи по индексу
    public void removeTask(int index) {
        model.removeTask(index);
    }

    // Отметить задачу как выполненную
    public void completeTask(int index) {
        model.completeTask(index);
    }

    // Получение списка всех задач
    public List<Task> getTasks() {
        return model.getTasks();
    }

    // Сохранение текущего списка задач в файл
    public void saveTasksToFile(String filename) {
        try (ObjectOutputStream out = new ObjectOutputStream(new
        FileOutputStream(filename))) {
            out.writeObject(model.getTasks());
        } catch (IOException e) {
            // Обработка исключений при записи в файл
            e.printStackTrace();
        }
    }

    // Загрузка списка задач из файла
    public void loadTasksFromFile(String filename) {
        try (ObjectInputStream in = new ObjectInputStream(new

```

```

FileInputStream(filename))) {
    List<Task> tasks = (List<Task>) in.readObject();
    model = new TaskModel(); // Создаем новую модель
    for (Task task : tasks) {
        model.addTask(task.getDescription());
        if (task.isCompleted()) {
            model.completeTask(model.getTaskCount() - 1);
        }
    }
} catch (IOException | ClassNotFoundException e) {
    // Обработка исключений при чтении из файла
    e.printStackTrace();
}
}
}

```

Класс TaskModelTest отвечает за тестирование модели. Исходный код модуля приведён ниже.

```

import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;

public class TaskModelTest {

    private TaskModel taskModel;

    @Before
    public void setUp() {
        taskModel = new TaskModel();
    }

    @Test
    public void testAddTask() {
        taskModel.addTask("Test task");
        assertEquals("Task count should be 1 after adding a task.", 1,
taskModel.getTaskCount());
    }

    @Test
    public void testRemoveTask() {
        taskModel.addTask("Test task");
        taskModel.removeTask(0);
        assertEquals("Task count should be 0 after removing the task.", 0,
taskModel.getTaskCount());
    }

    @Test
    public void testCompleteTask() {
        taskModel.addTask("Test task");
        taskModel.completeTask(0);
        assertTrue("Task should be marked as completed.",
taskModel.getTasks().get(0).isCompleted());
    }

    @Test
    public void testGetRemainingTasks() {
        taskModel.addTask("Test task 1");
        taskModel.addTask("Test task 2");
        taskModel.completeTask(0);
        assertEquals("There should be 1 remaining task.", 1,
taskModel.getRemainingTasks());
    }
}

```

```

    @Test
    public void testTaskDescription() {
        Task task = new Task("Test description");
        assertEquals("The description should match the one set in constructor.",
"Test description", task.getDescription());
    }

    @Test
    public void testSetDescription() {
        Task task = new Task("Initial description");
        task.setDescription("Updated description");
        assertEquals("The description should be updated.", "Updated description",
task.getDescription());
    }

    @Test
    public void testToString() {
        Task task = new Task("Test toString");
        assertEquals("toString should reflect the task is not completed.", "[ ]
Test toString", task.toString());
        task.setCompleted(true);
        assertEquals("toString should reflect the task is completed.", "[X] Test
toString", task.toString());
    }
}

```

Класс TaskControllerTest отвечает за тестирование контроллера. Исходный код модуля приведён ниже

```

import org.junit.Before;
import org.junit.Test;
import static org.junit.Assert.*;

public class TaskControllerTest {
    private TaskController controller;
    private TaskModel model;

    @Before
    public void setUp() {
        model = new TaskModel(); // предполагаем, что TaskModel - это класс модели
        controller = new TaskController(model); // предполагаем, что
TaskController принимает модель в конструкторе
    }

    @Test
    public void testAddTask() {
        controller.addTask("Новая задача");
        assertEquals("Количество задач должно быть 1 после добавления задачи", 1,
model.getTasks().size());
    }

    @Test
    public void testRemoveTask() {
        controller.addTask("Задача для удаления");
        controller.removeTask(0);
        assertTrue("Список задач должен быть пуст после удаления задачи",
model.getTasks().isEmpty());
    }
}

```

ОПИСАНИЕ

Главное окно программы представлено на рисунке 1.

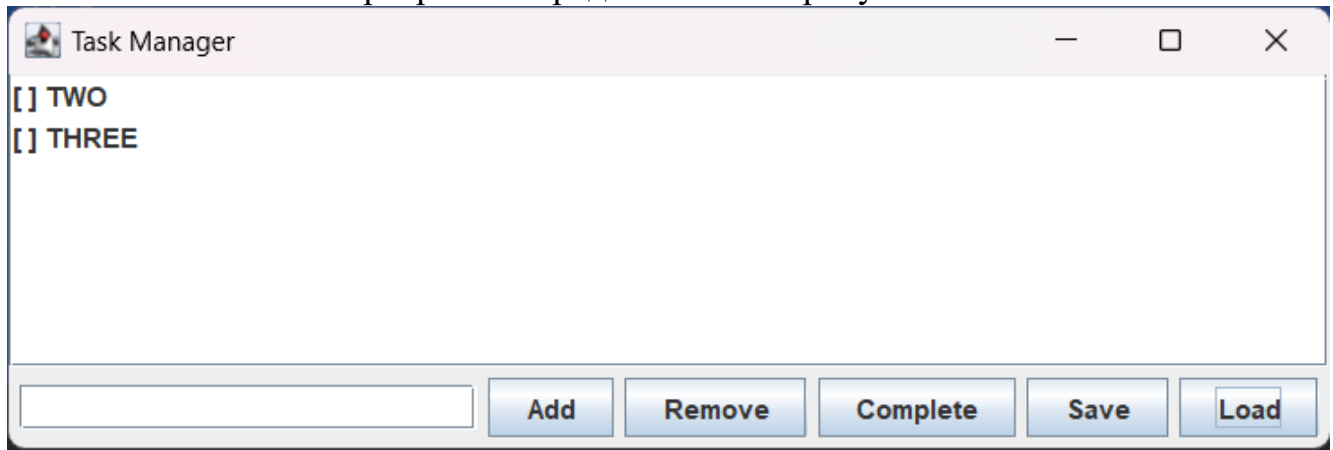


Рис.1 – Главное окно программы

Главное окно содержит кнопки «Add», «Remove», «Complete», «Save», «Load». Поле для ввода задачи.

Выполненная задача помечается «[X]» на рисунке 2.

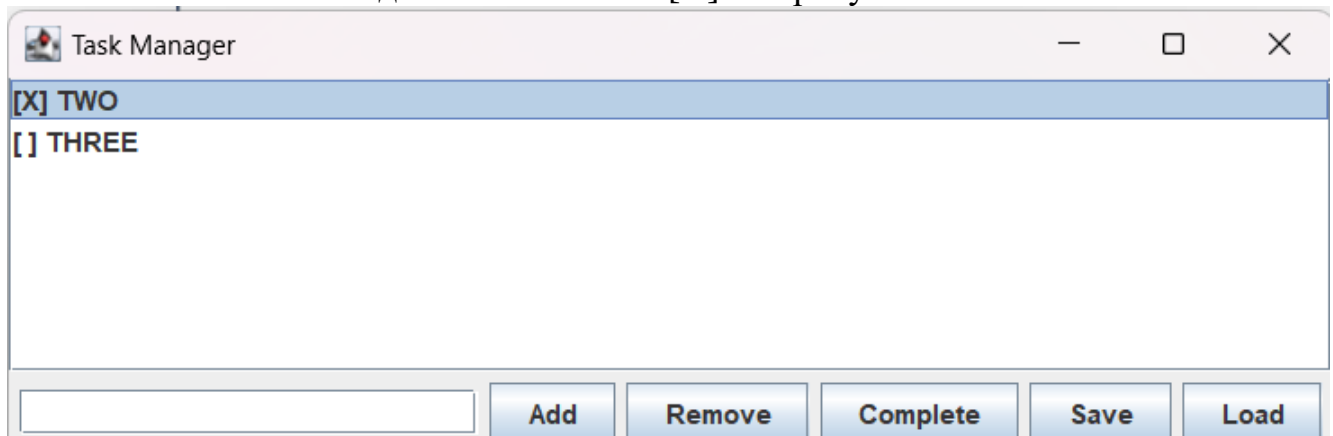


Рис.2 – Выполненная задача

Также задачу можно удалить. Перед выходом пользователь сохраняет сессию. Так при следующем запуске можно подгрузить задачи.

ОПИСАНИЕ ТЕСТОВ

Для валидации программы были разработаны следующие тесты.

Тест тестирует функцию удаления задачи.

```
@Test
public void testRemoveTask() {
    taskModel.addTask("Test task");
    taskModel.removeTask(0);
    assertEquals("Task count should be 0 after removing the task.", 0,
taskModel.getTaskCount());
}
```

Следующий тест проверяет маркировку выполненной задачи.

Исходный код теста приведён ниже.

```
@Test
public void testCompleteTask() {
    taskModel.addTask("Test task");
    taskModel.completeTask(0);
    assertTrue("Task should be marked as completed.",
taskModel.getTasks().get(0).isCompleted());
}
```

3-ий тест проверяет, что после добавления задачи кол-во задач увеличивается на 1.

Исходный код теста приведён ниже.

```
@Test
public void testAddTask() {
    taskModel.addTask("Test task");
    assertEquals("Task count should be 1 after adding a task.", 1,
taskModel.getTaskCount());
}
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была разработана программа, предназначена для ведения задачника, и система тестов для валидации разработанной программы. Были получены навык создания простых приложений на языке программирования Java, навык использования элементов для создания интерфейса.